

Diagnóstico: Problema de Acesso da LLM ao MCP

Problema Identificado

A LLM **externa** (xiaozi-mqtt-client) reportou: "**não conseguiu receber**" durante a inicialização MCP.

Logs Evidenciadores:

```
14:27:41,529[src.application] - INFO - Mensagem JSON recebida: type=llm - MainThread
```

A LLM recebeu uma mensagem, mas quando tentou acessar as capabilities do MCP (vision URL/token), falhou.

Análise Técnica

1. O que foi enviado corretamente:

```
{
  "jsonrpc": "2.0",
  "method": "initialize",
  "id": 1,
  "params": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "vision": {
        "url": "http://api.xiaozi.me/vision/explain",
        "token": "069448b6-bc70-423f-89ef-d930b53071b0"
      }
    }
  },
  "clientInfo": {
    "name": "xiaozi-mqtt-client",
    "version": "1.0.0"
  }
}
```

2. O que foi processado localmente:

- ☒ VL Camera inicializado com URL: <https://api.tenclass.net/xiaozi/vision/explain>
- ☒ 32 ferramentas MCP registradas
- ☒ Vision service configurado com URL <http://api.xiaozi.me/vision/explain>

- ☒ Foto capturada e analisada com sucesso (Ollama: 51 caracteres)
- ☒ 1204 frames de áudio processados
- ☒ Audio queue limpo em timing correto (2.5s delay)

3. Onde a LLM falhou:

A LLM externa **não conseguiu acessar** a URL de vision que foi configurada:

```
URL: http://api.xiaozhi.me/vision/explain
Token: 069448b6-bc70-423f-89ef-d930b53071b0
```

Causas Possíveis

Causa 1: URL de Visão Inacessível (MAIS PROVÁVEL)

- **✗** A URL `http://api.xiaozhi.me/vision/explain` pode estar:
 - Indisponível/fora do ar
 - Bloqueada por firewall
 - Exigindo VPN/proxy
 - Com token inválido/expirado

Causa 2: Falha de Timeout

- A LLM tentou acessar e aguardou resposta por muito tempo
- A requisição foi abortada por timeout

Causa 3: Erro de Autenticação

- Token inválido ou expirado
- Header `Authorization` mal formatado
- Endpoint esperando formato diferente de token

Causa 4: Incompatibilidade de Protocolo

- A LLM esperava HTTPS, recebeu HTTP
- Endpoint não está respondendo com JSON válido
- Versão do protocolo incompatível

☒ Soluções Recomendadas

Solução 1: Validar Acesso à URL de Visão (IMEDIATO)

```
# Teste a URL manualmente
curl -X POST http://api.xiaozhi.me/vision/explain \
  -H "Authorization: Bearer 069448b6-bc70-423f-89ef-d930b53071b0" \
```

```
-H "Content-Type: application/json" \
-d '{"image": "base64_image", "question": "teste"}'
```

Solução 2: Usar Ollama Local como Fallback (RECOMENDADO)

Trocar a visão externa por Ollama local é mais confiável:

```
# Em src/mcp/mcp_server.py - _parse_capabilities()

vision = capabilities.get("vision", {})
if vision and isinstance(vision, dict):
    url = vision.get("url")
    token = vision.get("token")

    if url:
        # Validar se URL é acessível antes de configurar
        try:
            import httpx
            response = httpx.head(url, timeout=5)
            if response.status_code == 200:
                camera.set_explain_url(url)
                logger.info(f"✅ Vision URL acessível: {url}")
            else:
                logger.warning(f"⚠️ Vision URL retornou {response.status_code}: {url}")
                logger.info("🔄 Usando Ollama local como fallback")
                # Usar Ollama: http://localhost:11434
                camera.set_explain_url("http://localhost:11434")
        except Exception as e:
            logger.error(f"❌ Erro ao validar Vision URL: {e}")
            logger.info("🔄 Usando Ollama local como fallback")
            camera.set_explain_url("http://localhost:11434")
```

Solução 3: Adicionar Retry Logic

```
# Em src/mcp/tools/camera/normal_camera.py

async def analyze_with_retry(self, question: str, max_retries: int = 3):
    """Tentar analisar com retry automático"""
    for attempt in range(max_retries):
        try:
            result = await self.analyze(question)
            return result
        except Exception as e:
            logger.warning(f"Tentativa {attempt+1} falhou: {e}")
            if attempt < max_retries - 1:
                await asyncio.sleep(2 ** attempt) # Backoff exponencial
            else:
```

```
# Última tentativa - usar fallback
logger.info("Usando Ollama local como fallback")
result = await self._analyze_with_ollama(...)
return result
```

Solução 4: Aumentar Timeout e Adicionar Health Check

```
# Em src/mcp/tools/providers/vllm_openai_provider.py

def __init__(self, config: Dict[str, Any]):
    # ... existing code ...

    # Adicionar timeout mais generoso
    self.client = openai.OpenAI(
        api_key=self.api_key,
        base_url=self.base_url,
        timeout=30.0, # Aumentar de padrão para 30s
        max_retries=3 # Adicionar retries automáticas
    )

    # Health check
    self._verify_connection()

def _verify_connection(self):
    """Verifica se o endpoint está acessível"""
    try:
        # Fazer test request simples
        response = self.client.models.list()
        logger.info(f"✅ Vision API acessível: {self.base_url}")
    except Exception as e:
        logger.error(f"❌ Vision API inacessível: {e}")
        logger.warning("Pode haver problemas durante execução")
```

Tabela de Diagnóstico

Componente	Status	Observação
MCP Server Inicialização	✅ OK	Recebeu capabilities
Camera VL Inicializada	✅ OK	URL configurada localmente
Foto Capturada	✅ OK	Ollama análise sucedida
Audio TTS	✅ OK	1204 frames, delays corretos
LLM Acesso à Vision URL	❌ FALHA	Não conseguiu receber/acessar
URL http://api.xiaozhi.me	? DESCONHECIDO	Precisa validação

Componente	Status	Observação
Token Autenticação	? DESCONHECIDO	Pode estar expirado

Próximas Ações

1. ☒ **Testar URL de visão** com curl ou Postman
2. ☒ **Verificar se token está válido** no painel de controle
3. ☒ **Implementar fallback para Ollama** (mais confiável)
4. ☒ **Adicionar health check** na inicialização
5. ☒ **Implementar retry logic** com backoff exponencial

Recomendação Final

Use Ollama local como provider padrão pois:

- ☒ 100% confiável (rodando localmente)
- ☒ Sem dependências de rede externas
- ☒ Sem limites de token
- ☒ Sem timeout de APIs externas
- ☒ Mais rápido (LAN vs internet)

A URL externa deveria ser apenas fallback para quando Ollama não estiver disponível.